Implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

CODE :

```
# EXPERIMENT 2: CANDIDATE-ELIMINATION ALGORITHM
import pandas as pd
# Step 1: Create the given dataset and save it as a CSV file
data = {
    'sky': ['sunny', 'sunny', 'rainy', 'sunny'],
    'airtemp': ['warm', 'warm', 'cold', 'warm'],
    'humidity': ['normal', 'high', 'high', 'high'],
    'wind': ['strong', 'strong', 'strong', 'strong'],
    'water': ['warm', 'warm', 'warm', 'cool'],
    'forcast': ['same', 'same', 'change', 'change'],
    'enjoysport': ['yes', 'yes', 'no', 'yes']
}
df = pd.DataFrame(data)
df.to_csv('/content/training_data.csv', index=False)
df = pd.read_csv('/content/training_data.csv')
print("TRAINING DATASET:")
print(df.to_string(index=False))
# Step 2: Prepare training examples
X = df.iloc[:, :-1].values.tolist()
y = df.iloc[:, -1].str.lower().tolist()
domains = [
    df[col].unique().tolist()
    for col in df.columns[:-1]
]
n = len(domains)
# Initial specific and general boundaries
S = [('Ø',) * n]
G = [('?',) * n]
# Check whether a hypothesis covers an example
def covers(h, x):
    return all(a == '?' or a == v for a, v in zip(h, x))
# Check whether h1 is at least as general as h2
def more_general_or_equal(h1, h2):
    return all(
```

```
        a == '?' or a == b or b == 'Ø'
        for a, b in zip(h1, h2)
    )

# Minimal generalization for positive examples
def generalize(h, x):
    result = list(h)
    for i in range(n):
        if result[i] == 'Ø':
            result[i] = x[i]
        elif result[i] != x[i]:
            result[i] = '?'
    return tuple(result)

# Minimal specialization for negative examples
def specialize(h, x, S_boundary):
    result = []

    for i in range(n):
        if h[i] == '?':
            for value in domains[i]:
                if value != x[i]:
                    candidate = list(h)
                    candidate[i] = value
                    candidate = tuple(candidate)

                    if any(
                        more_general_or_equal(candidate, s)
                        for s in S_boundary
                    ):
                        result.append(candidate)

    return result

# Step 3: Process each training example
for example, label in zip(X, y):
    x = tuple(example)

    if label == 'yes':
        # Remove G hypotheses that do not cover the positive example
```

```
        G = [g for g in G if covers(g, x)]

        # Generalize S to cover the positive example
        new_S = []
        for s in S:
            if covers(s, x):
                new_S.append(s)
            else:
                new_S.append(generalize(s, x))

        S = list(set(
            s for s in new_S
            if any(more_general_or_equal(g, s) for g in G)
        ))

        # Retain maximally specific hypotheses
        S = [
            s for s in S
            if not any(
                s != other and more_general_or_equal(s, other)
                for other in S
            )
        ]

    else:
        # Remove S hypotheses that cover the negative example
        S = [s for s in S if not covers(s, x)]

        # Specialize G to reject the negative example
        new_G = []

        for g in G:
            if not covers(g, x):
                new_G.append(g)
            else:
                new_G.extend(specialize(g, x, S))

        G = list(set(
            g for g in new_G
            if any(more_general_or_equal(g, s) for s in S)
```

```
        ))

        # Retain maximally general hypotheses
        G = [
            g for g in G
            if not any(
                g != other and more_general_or_equal(other, g)
                for other in G
            )
        ]

    print(f"\nTraining Example: {x} -> {label.upper()}")
    print("Specific Boundary S:", S)
    print("General Boundary G:", G)

# Step 4: Display final boundaries
print("\n========== FINAL RESULT ==========")

print("\nFinal Specific Boundary (S):")
for h in S:
    print(h)

print("\nFinal General Boundary (G):")
for h in G:
    print(h)

print("\nResult: Candidate-Elimination algorithm executed successfully.")
```

Outcome :

```
========== FINAL RESULT ==========

Final Specific Boundary (S):
('sunny', 'warm', '?', 'strong', '?', '?')

Final General Boundary (G):
('?', 'warm', '?', '?', '?', '?')
('sunny', '?', '?', '?', '?', '?')

Result: Candidate-Elimination algorithm executed successfully.
```